# JAVASCRIPT HANDWRITTEN NOTES

## Part 1

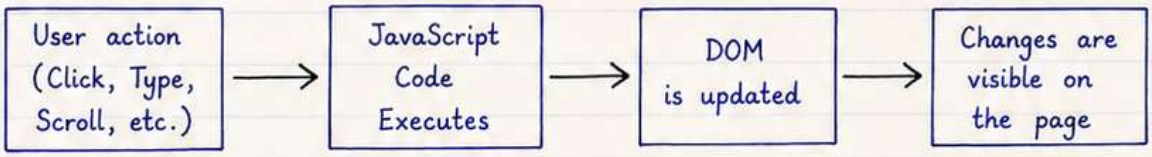
## 1. INTRODUCTION TO JAVASCRIPT

- **What is JavaScript ?**
  → JavaScript is a lightweight, interpreted programming language used to make web pages interactive and dynamic.
  It is one of the core technologies of the web along with HTML and CSS.
- **Why JavaScript ?**
  → It allows us to add behavior to web pages.
  It can update and change both HTML and CSS.
- **Where does JavaScript run ?**
  → In the Browser (Client-side)
  → On the Server using Node.js (Server-side)
- **Role of JavaScript in Web Development**

| HTML | → | CSS | → | JavaScript |
|---|---|---|---|---|
| Structure of the page | | Style and Design | | Behavior and Interactivity |

} Together they build a complete web page.

## 2. HOW JAVASCRIPT WORKS ?




Note : DOM (Document Object Model) is a representation of the HTML structure that JavaScript can access and manipulate.

## 3. YOUR FIRST JAVASCRIPT CODE

- **Example :**

```
<script>
    console.log("Hello, JavaScript!");
</script>
```

→ This code prints the message in the browser console.

- **Output in Console :**

```
Hello, JavaScript!
```

→ You can check the output by opening browser console.

## 1. VARIABLES IN JAVASCRIPT

- Variables are used to store data values.
- In JavaScript, we can declare variables using var, let and const.

| var | let | const |
|---|---|---|
| • Function scoped<br>• Can be re-declared<br>• Can be updated<br>• Hoisted with undefined | • Block scoped<br>• Cannot be re-declared<br>• Can be updated<br>• Hoisted but in Temporal Dead Zone (TDZ) | • Block scoped<br>• Cannot be re-declared<br>• Cannot be updated<br>• Must be initialized at the time of declaration |

Example :

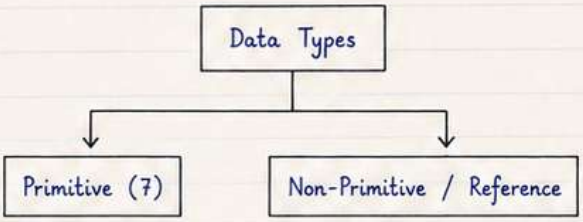
```
var a = 10;
let b = 20;
const c = 30;

a = 15;      // allowed
b = 25;      // allowed
c = 35;      // not allowed
```

NOTE : Prefer using let and const instead of var.

## 2. DATA TYPES IN JAVASCRIPT

Data types are basically used to define the type of value a variable can hold.




- String
- Number
- Boolean
- Undefined
- Null
- Symbol (ES6)
- BigInt (ES2020)

- Object
- Array
- Function

Example :

```
let name = "Aman";                    // String
let age = 22;                         // Number
let isStudent = true;                 // Boolean
let x;                                // Undefined
let y = null;                         // Null
let id = Symbol('id');                // Symbol
let big = 1234567890123456789012345678901234567890n; // BigInt
let obj = {name: "Aman"};             // Object
let arr = [1, 2, 3];                  // Array
function greet() {}                   // Function
```

## 3. TYPEOF OPERATOR

typeof operator is used to find the type of a variable.

Example :

```
console.log(typeof "Aman");          // string
console.log(typeof 22);              // number
console.log(typeof true);            // boolean
console.log(typeof undefined);       // undefined
console.log(typeof null);            // object
console.log(typeof {name:"Aman"});   // object
console.log(typeof [1,2,3];          // object
console.log(typeof function(){});    // function
```

Note :

- typeof null returns "object" (this is a known bug in JavaScript).
- Arrays and Objects both return "object".

# 4. OPERATORS IN JAVASCRIPT

- Operators are used to perform operations on variables and values.

## Types of Operators :

| Type | Description | Example |
|---|---|---|
| Arithmetic | Used for mathematical operations | + , - , * , / , % |
| Assignment | Used to assign values | = , += , -= , *= , /= |
| Comparison | Used to compare two values | == , != , > , < , >= , <= |
| Logical | Used to combine multiple conditions | && , \|\| , ! |
| Unary | Used with a single operand | ++ , -- , ! |
| Ternary | Short-hand if...else | condition ? expr1 : expr2 |

## Arithmetic Operators :

| | |
|---|---|
| + | Addition |
| - | Subtraction |
| * | Multiplication |
| / | Division |
| % | Modulus (remainder) |
| ++ | Increment |
| -- | Decrement |

Example :

```
let a = 10, b = 3;
console.log(a + b);      // 13
console.log(a - b);      // 7
console.log(a * b);      // 30
console.log(a / b);      // 3.333...
console.log(a % b);      // 1
a++;                     // 11
b--;                     // 2
```

## Comparsion Operators :

| | |
|---|---|
| == | Equal to |
| != | Not equal to |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal to |
| <= | Less than or equal to |
| === | Strict equal (value and type) |
| !== | Strict not equal (value and type) |

Example :

```
console.log(5 == "5");      // true
console.log(5 === "5");     // false (type different)
console.log(5 != 3);        // true
console.log(5 > 3);         // true
console.log(5 < 3);         // false
console.log(5 >= 5);        // true
console.log(5 <= 4);        // false
```

## Logical Operators :

| | |
|---|---|
| && | Logical AND |
| \|\| | Logical OR |
| ! | Logical NOT |

Example :

```
console.log(true && false);  // false
console.log(true || false);  // true
console.log(!true);          // false
```

## Ternary Operator :

```
condition ? expr1 : expr2
```

Example :

```
let age = 18;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result);    // Adult
```

## 5. CONTROL STATEMENTS

- These statements are used to control the flow of execution.

a) if statement

```
if (condition) {
    // code
}
```

b) if...else statement

```
if (condition) {
    // code if true
} else {
    // code if false
}
```

c) if...else if...else ladder

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else {
    // code
}
```

d) switch statement

```
switch (variable) {
  case value1:
      // code
      break;
  case value2:
      // code
      break;
  default:
      // code
}
```

e) Loops

- for loop

```
for (initialization; condition; increment/decrement){
    // code
}
```

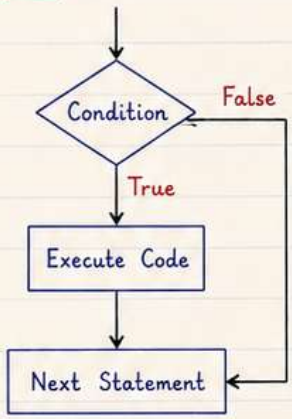
- while loop

```
while (condition) {
    // code
}
```

- do...while loop

```
do {
    // code
} while (condition);
```

Flow :




## 6. FUNCTIONS IN JAVASCRIPT

- A function is a block of code designed to perform a particular task.

- Function Declaration

```
function greet(name) {
    console.log("Hello, " + name);
}
greet("Aman");  // Function Call
```

- Function Expression

```
const greet = function(name) {
    console.log("Hello, " + name);
};
greet("Aman");
```

- Arrow Function

```
const greet = (name) => {
    console.log("Hello, " + name);
};
greet("Aman");
```

Note : Functions help in code reusability and keep our code organized.

## 7. COMMENTS IN JAVASCRIPT

```
// This is a single line comment.
```

```
/*
This is
a multi-line
comment.
*/
```

Example :

```
console.log(10);    // prints 10
console.log(20);    /* prints 20 on
                       two line comment */
console.log(30);    // prints 30
```

## 8. ARRAYS IN JAVASCRIPT

- Array is a special type of object used to store multiple values in a single variable.
- Index of an array starts from 0.

Example :

```
let fruits = ["Apple", "Banana", "Mango"];
```

| index → | 0 | 1 | 2 |
|---|---|---|---|
| | Apple | Banana | Mango |

Common Array Methods :

- push() → adds element at the end
- pop() → removes last element
- shift() → removes first element
- unshift() → adds element at the beginning
- length → returns number of elements
- indexOf() → returns index of element

Accessing Elements :

- fruits[0] → Apple
- fruits[1] → Banana
- fruits[2] → Mango

Example :

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.push("Orange");        // ["Apple","Banana","Mango","Orange"]
fruits.pop();                 // ["Apple","Banana","Mango"]
console.log(fruits[1]);       // Banana
```

## 9. OBJECTS IN JAVASCRIPT

- Object is a collection of key-value pairs.
- It is used to store related data and more complex entities.

Example :

```
let person = {
    name: "Aman",
    age: 22,
    city: "Delhi"
};
```

Accessing Object Properties :

- person.name → Aman
- person["age"] → 22
- person.city → Delhi

Example :

```
let person = { name: "Aman", age: 22, city: "Delhi" };
console.log(person.name);     // Aman
console.log(person["age"]);   // 22
```

## 10. TYPE CONVERSION

- JavaScript automatically converts one type into another.
- Two types : Implicit Conversion and Explicit Conversion

Implicit Conversion (Automatic)

- "5" + 2 → "52" (string)
- "5" - 2 → 3 (number)
- true + 1 → 2 (number)
- false + 1 → 1 (number)
- null + 1 → 1 (number)
- undefined + 1 → NaN (Not a Number)

Explicit Conversion (Manual)

- String to Number : Number("123") → 123
- Number to String : String(123) → "123"
- Boolean to String : String(true) → "true"
- String to Number : parseInt("45") → 45
  parseFloat("45.67") → 45.67

Example :

```
let a = "10";
console.log(typeof a);            // string
console.log(typeof Number(a));    // number
```

```
let b = 20;
console.log(typeof String(b));    // string
console.log(typeof (a + b));      // string ("1020")
```

## 11. STRING METHODS

- Strings are used to represent and manipulate text.

Common String Methods :

| Method | Description |
|---|---|
| length | Returns length of the string |
| toUpperCase() | Converts string to uppercase |
| toLowerCase() | Converts string to lowercase |
| trim() | Removes whitespace from both ends |
| slice(start, end) | Extracts part of a string |
| replace(old, new) | Replaces a value in string |
| includes(value) | Checks if string contains value |

Example :

```
let msg = "Hello JavaScript";
console.log(msg.length);          // 16
console.log(msg.toUpperCase()); // HELLO JAVASCRIPT
console.log(msg.toLowerCase()); // hello javascript
console.log(msg.trim());          // "Hello JavaScript"
console.log(msg.slice(6,10));    // Java
console.log(msg.replace("JavaScript","JS")); // Hello JS
console.log(msg.includes("Java")); // true
```

## 12. TEMPLATE LITERALS

- Template literals allow embedded expressions and multi-line strings.

Example :

```
let name = "Aman";
let age = 22;
let msg = `
    Hello, ${name}!
    You are ${age} years old.
`;
console.log(msg);
```

Output :

```
Hello, Aman!
You are 22 years old.
```

Note :

Use backtick (`) instead of single or double quotes.

Use ${ } to embed variables or expressions.

## 13. CONDITIONAL (TERNARY) OPERATOR

- Short hand if...else statement.

```
condition ? expression_if_true : expression_if_false
```

Example :

```
let age = 18;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result);  // Adult
```

## 14. EVENTS IN JAVASCRIPT

- Events are actions or occurrences that happen in the browser.
- JavaScript can be used to handle these events.

Common Events :

| click | mouseover | mouseout | keydown | submit |
|---|---|---|---|---|
| Occurs when an element is clicked. | Occurs when mouse pointer moves over. | Occurs when mouse pointer moves out. | Occurs when a key is pressed. | Occurs when a form is submitted. |

Example (click event) :

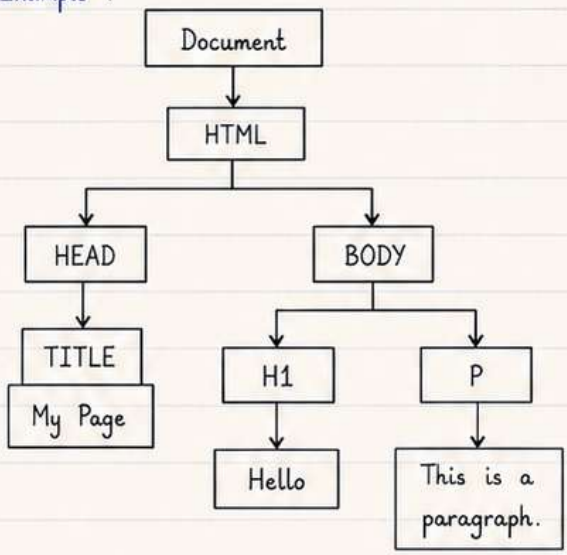
```
document.getElementById("btn").
    addEventListener("click", function() {
        alert("Button Clicked!");
});
```

HTML : `<button id="btn">Click Me</button>`

## 15. DOM (DOCUMENT OBJECT MODEL)

- DOM represents the structure of an HTML document as a tree.
- JavaScript can access and update the content, structure and style of the page using the DOM.

Example :




Accessing DOM Elements :

- document.getElementById() → by id
- document.getElementsByClassName() → by class
- document.getElementsByTagName() → by tag
- document.querySelector() → by css selector
- document.querySelectorAll() → all matching

Example :

```
<h1 id = "title" > Hello DOM </h1>
<p class = "text" > Welcome </p>

let h1 = document.getElementById("title");
let p = document.getElementsByClassName("text")[0];
h1.innerText = "Hello JavaScript";
p.innerText = "Welcome to JS";
```

## 16. EVENT HANDLING

- Events are actions that occur in the browser.
- We can respond to events using event listeners.

Common Events :

| Event | Description |
|---|---|
| click | Occurs when an element is clicked |
| mouseover | Occurs when mouse is over an element |
| mouseout | Occurs when mouse is moved out |
| keydown | Occurs when a key is pressed |
| submit | Occurs when a form is submitted |

Example (click event) :

```
<button id = "btn" > Click Me </button>
<p id = "msg" ></p>

document.getElementById("btn").addEventListener
("click", function() {
    document.getElementById("msg").innerText =
        "Button Clicked!";
});
```

## 17. COMMENTS IN JAVASCRIPT

a) Single-line Comment

```
// This is a single line comment
```

b) Multi-line Comment

```
/*
  This is
  a multi-line comment
  */
```

Note :

Comments are ignored by JavaScript and are used to explain the code.

## 18. SCOPE IN JAVASCRIPT

- Scope determines the accessibility (visibility) of variables.
- JavaScript has 3 types of scope.

Types of Scope :

1. Global Scope – Declared outside any function, accessible everywhere.
2. Function Scope – Declared inside a function, accessible only inside that function.
3. Block Scope – Declared inside { } using let or const, accessible only inside the block.

1. Global Scope

```
let a = 10;          // global
function test() {
    console.log(a);  // 10
}
test();
console.log(a);      // 10
```

2. Function Scope

```
function test() {
    let b = 20;      // function scope
    console.log(b);  // 20
}
test();
// console.log(b);   // Error (b is
                     not accessible here)
```

3. Block Scope (let/const)

```
if (true) {
    let c = 30;      // block scope
    console.log(c);  // 30
}
// console.log(c);   // Error (c is
       not accessible here)
```

## 19. DATA TYPES IN JAVASCRIPT

- JavaScript has 7 primitive data types and 1 non-primitive data type (Object).

Primitive Data Types (7) :

1. String → "Hello"
2. Number → 123, 45.67
3. Boolean → true, false
4. Undefined → let a;
5. Null → let a = null;
6. Symbol → let s = Symbol("id");
7. BigInt → let n = 9007199254740991n;

Non-Primitive Data Type (1) :

- Object → { key: "value" }

Example :

```
let person = {
    name: "Aman",
    age: 22
};
console.log(typeof person); // object
```

## 20. TYPEOF OPERATOR

- typeof operator is used to find the data type of a value.

Example :

| Value | typeof Value | Result |
|---|---|---|
| "Hello" | typeof "Hello" | "string" |
| 123 | typeof 123 | "number" |
| true | typeof true | "boolean" |
| undefined | typeof undefined | "undefined" |

| Value | typeof Value | Result |
|---|---|---|
| null | typeof null | "object" (special case) |
| Symbol("id") | typeof Symbol("id") | "symbol" |
| 900n | typeof 900n | "bigint" |
| { } | typeof { } | "object" |

## 21. VARIABLES IN JAVASCRIPT

- Variables are used to store data values.
- JavaScript has 3 ways to declare variables.

1. var (old way)

```
var a = 10;
// function scoped
```

2. let (modern way)

```
let b = 20;
// block scoped
```

3. const (constant)

```
const c = 30;
// cannot be reassigned
```

## 22. VARIABLES

- Variables are containers used to store data values.
- Rules for naming variables :

1. Must start with a letter, $ or _
2. Cannot start with a number.
3. Can contain letters, numbers, $ and _
4. Case sensitive (age and Age are different)
5. Cannot be a JavaScript keyword.

Example :

| Valid Variables | Invalid Variables |
|---|---|
| name | 1name (starts with number) |
| _name | user-name (contains -) |
| $price | let (keyword) |
| age1 | first name (contains space) |
| user_name | @value (contains @) |

## 23. ARITHMETIC OPERATORS

| Operator | Name | Example | Result |
|---|---|---|---|
| + | Addition | 5 + 3 | 8 |
| - | Subtraction | 5 - 3 | 2 |
| * | Multiplication | 5 * 3 | 15 |
| / | Division | 10 / 2 | 5 |
| % | Modulus | 10 % 3 | 1 |
| ++ | Increment | a++ | a + 1 |
| -- | Decrement | a-- | a - 1 |

Example :

```
let a = 10, b = 3;
console.log(a + b);      // 13
console.log(a - b);      // 7
console.log(a * b);      // 30
console.log(a / b);      // 3.333...
console.log(a % b);      // 1
a++;
console.log(a);          // 11
b--;
console.log(b);          // 2
```

## 24. COMPARISON OPERATORS

| Operator | Name | Example | Result |
|---|---|---|---|
| == | Equal to | 5 == 5 | true |
| != | Not equal to | 5 != 3 | true |
| === | Strict equal to | 5 === "5" | false |
| !== | Strict not equal | 5 !== "5" | true |
| > | Greater than | 5 > 3 | true |
| < | Less than | 5 < 3 | false |
| >= | Greater than or equal | 5 >= 5 | true |
| <= | Less than or equal | 5 <= 4 | false |

Example :

```
let x = 5, y = "5";
console.log(x == y);      // true
console.log(x === y);     // false
console.log(x != y);      // false
console.log(x !== y);     // true
console.log(x > 3);       // true
console.log(x <= 5);      // true
```

## 25. LOGICAL OPERATORS

| Operator | Name | Description | Example |
|---|---|---|---|
| && | AND | Returns true if both conditions are true | (5 > 3 && 3 > 1) |
| \|\| | OR | Returns true if at least one condition is true | (5 > 3 \|\| 3 < 1) |
| ! | NOT | Reverses the result | !(5 > 3) |

Example :

```
let a = 5, b = 3, c = 1;
console.log(a > b && b > c);    // true
console.log(a < b || c < b);    // true
console.log(!(a < b));          // true
```

## 26. CONTROL FLOW STATEMENTS

- Control flow statements are used to control the flow of execution based on conditions.

Types :

| 1. if Statement | 2. if...else | 3. else if Ladder | 4. switch Statement | 5. ternary Operator |
|---|---|---|---|---|
| Executes a block if condition is true. | Executes one block if condition is true, otherwise another. | Checks multiple conditions one by one. | Executes a block based on different cases. | Short hand if...else used for simple conditions. |

Example :

**1. if**

```
let x = 10;
if (x > 5) {
  console.log("X is
  greater than 5");
}
```

**2. if...else**

```
let x = 3;
if (x > 5) {
  console.log("X > 5");
} else {
  console.log("X <= 5");
}
```

**3. else if Ladder**

```
let x = 7;
if (x > 10) {
  console.log("GT 10");
} else if (x > 5) {
  console.log("GT 5");
} else {
  console.log("<= 5");
}
```

**4. switch**

```
let day = 2;
switch(day) {
 case 1: console.log("Mon");
         break;
 case 2: console.log("Tue");
         break;
 default: console.log("Other");
}
```

**5. ternary**

```
let age = 18;
let result = (age >= 18)
      ? "Adult"
      : "Minor";
console.log(result);
```

## 27. LOOPS IN JAVASCRIPT

- Loops are used to execute a block of code multiple times.
- JavaScript has 3 types of loops.

Types :

**1. for Loop**

Used when number of iterations is known.

Example :

```
for (let i = 1; i <= 5; i++) {
  console.log(i);  // 1 to 5
}
```

**2. while Loop**

Executes the block as long as the condition is true.

Example :

```
let i = 1;
while (i <= 5) {
  console.log(i);  // 1 to 5
  i++;
}
```

**3. do...while Loop**

Executes the block at least once, then checks the condition.

Example :

```
let i = 1;
do {
  console.log(i);  // 1 to 5
  i++;
} while (i <= 5);
```

## 28. FUNCTIONS IN JAVASCRIPT

- A function is a block of code designed to perform a particular task.
- It helps in reusability and organization of code.

**1. Function Declaration**

```
function add(a, b) {
  return a + b;
}
let sum = add(5, 3);
console.log(sum);   // 8
```

**2. Function Expression**

```
let multiply = function(a, b) {
  return a * b;
};
let result = multiply(4, 2);
console.log(result);   // 8
```

**3. Arrow Function (ES6)**

```
let divide = (a, b) => a / b;

let res = divide(10, 2);
console.log(res);   // 5
```

**4. Default Parameter**

```
function greet(name = "Guest") {
  console.log("Hello, " + name);
}
greet();          // Hello, Guest
greet("Aman");    // Hello, Aman
```

**5. Rest Parameter (ES6)**

```
function sumAll(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
console.log(sumAll(1, 2, 3, 4));  // 10
```

**6. Immediately Invoked Function Expression (IIFE)**

```
(function() {
  console.log("IIFE executed");
})();

// Runs immediately
```

# 29. FUNCTIONS IN JAVASCRIPT

- Functions are reusable blocks of code that perform a specific task.
- They help in code reusability and modularity.

### Function Syntax :

```
function functionName(parameters) {
    // code to be executed
    return value;  // optional
}
```

### Example :

```
function add(a, b) {
    let sum = a + b;
    return sum;
}
let result = add(5, 3);
console.log(result);   // 8
```

### How it Works :

1. Define the function using the function keyword.
2. Pass arguments when calling the function.
3. Code inside the function executes.
4. If return is used, the value is sent back.

### Types of Functions :

#### 1. Function Declaration

```
function greet(name) {
    return "Hello " + name;
}
console.log(greet("Aman"));
// Hello Aman
```

- Defined using function keyword.
- Can be called before declaration.

#### 2. Function Expression

```
const greet = function(name) {
    return "Hello " + name;
};
console.log(greet("Aman"));
// Hello Aman
```

- Assigned to a variable.
- Cannot be called before declaration.

#### 3. Arrow Function (ES6)

```
const greet = (name) => {
    return "Hello " + name;
};
console.log(greet("Aman"));
// Hello Aman
```

- Shorter syntax.
- Does not have its own this.

### Parameters vs Arguments :

- Parameters are variables listed in the function definition.
- Arguments are values passed when calling the function.

| Parameters (in definition) | Arguments (in call) |
|---|---|
| `function multiply(x, y) {`<br>`  return x * y;`<br>`}` | `let result = multiply(4, 5);`<br>`console.log(result);  // 20` |

### Return Statement :

- return is used to send a value back from the function.
- If no return is used, function returns undefined.

```
function square(n) {
    return n * n;
}
console.log(square(4));
// 16
```

```
function print() {
    console.log("Hi");
}
let val = print();
console.log(val);   // undefined
```

### Default Parameters (ES6) :

```
function greet(name = "Guest") {
    return "Hello, " + name;
}
console.log(greet());          // Hello, Guest
console.log(greet("Aman"));    // Hello, Aman
```

### Rest Parameters (ES6) :

```
function sumAll(...nums) {
    return nums.reduce((acc, val) => acc + val, 0);
}
console.log(sumAll(1, 2, 3));          // 6
console.log(sumAll(1, 2, 3, 4, 5));    // 15
```

# 30. FUNCTIONS IN JAVASCRIPT (CONT.)

- Functions can be created in different ways.
- They can have parameters, return values and can be used multiple times.

## Other Ways to Create Functions :

### 1. Function Expression

```
const add = function(a, b) {
    return a + b;
}
console.log(add(2, 3));   // 5
```

### 2. Arrow Function (ES6)

```
const add = (a, b) => {
    return a + b;
}
console.log(add(2, 3));   // 5
```

### 3. Arrow Function (Shorter)

```
const add = (a, b) => a + b;

console.log(add(2, 3));   // 5
```

### 4. Immediately Invoked Function Expression (IIFE)

```
(function() {
    console.log("IIFE executed");
})();                                  // Runs immediately
```

## Function with Default Parameter (ES6) :

```
function greet(name = "Guest") {
    return "Hello, " + name;
}
console.log(greet());          // Hello, Guest
console.log(greet("Aman"));    // Hello, Aman
```

## Function with Rest Parameter (ES6) :

```
function sumAll(...nums) {
    return nums.reduce((acc, val) => acc + val, 0);
}
console.log(sumAll(1, 2, 3));          // 6
console.log(sumAll(1, 2, 3, 4, 5));    // 15
```

## Return Statement :

- return is used to send a value back from the function.
- If no return is used, function returns undefined.

```
function square(n) {
    return n * n;
}
console.log(square(4));  // 16
```

```
function show() {
    console.log("Hi");
}
let val = show();
console.log(val);   // undefined
```

## Function Rules / Points to Remember :

1. Function name should follow variable naming rules.
2. Function names are case-sensitive.
3. Function can be passed as a value.
4. Function can return another function.
5. Anonymous functions can be used.

## Example : Function returning another function

```
function outer() {
    return function inner() {
        return "Hello from inner";
    }
}
const fn = outer();
console.log(fn());   // Hello from inner
```

# FOLLOW FOR PART 2

MORE ADVANCED TOPICS,
EXAMPLES & REAL-WORLD PROJECTS AHEAD!

# COMMENT JAVASCRIPT

FOR PDF NOTES

JS

FOR PART 2
TARGET IS 10K❤️